

BREAKING APP

Admin Panel (Web Dashboard) — AI Development Prompt Document

DOCUMENT PURPOSE

This document is a comprehensive, structured prompt for an AI coding assistant to build the Admin Web Dashboard for 'Breaking App'. The admin panel allows administrators to manage all content and settings that power the mobile app. Follow every section precisely. Do not skip any feature.

1. PROJECT OVERVIEW

Build a web-based Admin Dashboard for 'Breaking App' using React.js or Next.js. This panel is used by administrators to create and manage news, categories, hashtags, cities, advertisements, users, and analytics. It communicates with the same REST API backend used by the mobile app.

 Use Next.js (App Router) as the preferred framework for SSR benefits. Alternatively React.js (Vite) is acceptable.

2. TECH STACK

- Framework: Next.js (preferred) or React.js
- Language: TypeScript
- Styling: Tailwind CSS + shadcn/ui component library
- State Management: Zustand or Redux Toolkit
- HTTP Client: Axios with interceptors for auth token
- Rich Text Editor: Quill.js or TipTap (for news content)
- Drag & Drop: @dnd-kit/core (for news ordering)
- Charts / Analytics: Recharts or Chart.js
- Media Upload: Direct upload to AWS S3 via pre-signed URLs
- Authentication: JWT-based login (token stored in httpOnly cookie or localStorage)
- Date/Time: Day.js for formatting
- Color Picker: react-colorful (for category colors)

3. AUTHENTICATION

3.1 Login Page

- Route: /login
- Fields: Email/Username, Password
- On success: store JWT token, redirect to dashboard
- On failure: show error message
- 'Forgot password' link (optional, placeholder is acceptable)

3.2 Auth Protection

- All routes except /login are protected
 - If token is missing or expired, redirect to /login
 - Implement token refresh if the backend supports it
-

4. LAYOUT & NAVIGATION

Use a standard admin dashboard layout:

- Fixed left sidebar with navigation links
- Top header bar with: App logo/name, logged-in admin name & avatar, Logout button
- Main content area (right of sidebar)

Sidebar navigation links:

- Dashboard (Analytics)
 - News Management
 - Category Management
 - Hashtag Management
 - City Management
 - Advertisement Management
 - News Ordering
 - Push Notifications
 - Admin Users
 - Settings
-

5. PAGES & FEATURES

5.1 Dashboard — Analytics

Route: /dashboard

Display key metrics in stat cards at the top:

- Total App Users
- Total News Views
- Total Shares
- Total Saved / Bookmarked News

Below the stat cards, show charts:

- Bar chart: Category-wise news performance (views per category)
- Table: Most Viewed News (top 10, with title, views, date)
- Table: Most Shared News (top 10, with title, shares, date)

 All analytics data is fetched from `GET /api/admin/analytics`

5.2 News Management

Route: `/news`

News List Page:

- Paginated table of all news posts
- Columns: Thumbnail, Title, Category, Status (Published/Draft), Date, Actions
- Actions per row: Edit, Delete, Toggle Publish/Unpublish
- Filters at top: by Category, by Status, by Date range
- Search bar: search by title
- 'Create News' button (navigates to Create page)

Create / Edit News Page:

- Route: `/news/create` and `/news/edit/:id`
- Field: News Title (text input, required)
- Field: Description / Content (Rich Text Editor — supports bold, italic, lists, links)
- Field: Category (dropdown, fetched from API)
- Field: Hashtags (multi-select or tag input, fetched from API)
- Field: City (multi-select dropdown, fetched from API)
- Field: Media Upload — drag-and-drop or browse; supports multiple images and videos; uploaded to AWS S3; preview shown after upload; support reordering and deletion of uploaded media
- Field: Reporter/Admin name (text input or select from admin users)
- Field: Reporter profile image (upload or auto-filled from selected admin)
- Field: Publish Date (date-time picker; allows scheduling)
- Field: Status toggle — Published / Draft
- Buttons: 'Save as Draft', 'Publish', 'Cancel'

 When editing, pre-fill all fields with existing news data fetched from `GET /api/news/:id`

5.3 Category Management

Route: `/categories`

- Table listing all categories with columns: Name, Status (Active/Inactive), Card BG Color, Font Color, Actions
- Actions: Edit, Delete, Toggle Active/Inactive
- 'Add Category' button opens a modal or form with:
 - Category Name (text input)
 - Background Color (color picker)
 - Font Color (color picker)
 - Active/Inactive toggle
- Edit category opens the same form pre-filled
- Confirm dialog before delete

5.4 Hashtag Management

Route: /hashtags

- Table listing all hashtags with columns: Hashtag Name, Actions
- Actions: Edit, Delete
- 'Add Hashtag' button opens modal with hashtag name field (auto-prefix # if not entered)
- Edit opens same modal pre-filled
- Confirm dialog before delete

5.5 City Management

Route: /cities

- Table listing all cities
- 'Add City' button with city name input
- Edit and Delete actions per row
- Confirm dialog before delete

5.6 Advertisement Management

Route: /advertisements

- Table listing all ads with columns: Preview thumbnail, Title/Name, Position, Status, Redirect URL, Actions
- Actions: Edit, Delete, Toggle Enable/Disable
- 'Add Advertisement' button opens modal/form with:
 - Ad name / label
 - Banner image upload (to AWS S3, preview shown)
 - Redirect URL (clickable link on mobile)
 - Position number (numeric — e.g., position 4 = after 4th news card)
 - Enable / Disable toggle
- Default position: after every 4th news card
- Multiple ads can be created at different positions
- Ad sequence/order is manageable

5.7 News Position / Ordering

Route: /news-ordering

- Display list of published news items
- Drag-and-drop to reorder (use @dnd-kit/core)
- 'Pin to Top' option for each news item (forces it to appear first in the feed)
- 'Schedule as Featured' option (set date/time for a news to be featured)
- 'Save Order' button to persist the order via API

5.8 Push Notifications

Route: /notifications

Send Notification Form:

- Title (text input)
- Message / Body (text area)
- Target: All Users / Specific Category
- Schedule: Send Now OR pick a date-time for scheduled send
- 'Send' button — calls POST /api/admin/notifications/send

Notification History:

- Table of past sent notifications with columns: Title, Message, Sent At, Target, Status

5.9 Admin User Management

Route: /admin-users (visible to Super Admin only)

- Table listing all admin users: Name, Email, Role, Status, Actions
 - Actions: Edit, Delete, Enable/Disable
 - 'Add Admin' button opens form with:
 - Name
 - Email
 - Mobile number
 - Password (set initial password)
 - Role (Super Admin / Reporter / Editor)
 - Profile image upload
 - Permissions checkboxes: can create news, can delete news, can manage categories, etc.
 - Edit opens same form pre-filled
 - Confirm dialog before delete
-

6. API INTEGRATION

All data is fetched from the REST API. Use an Axios instance with a base URL from environment variables. Attach the JWT Bearer token to every request via an Axios request interceptor.

Endpoint	Method	Purpose
/api/admin/login	POST	Admin login — returns JWT token
/api/admin/news	GET / POST	List all news (with filters) / Create new news post
/api/admin/news/:id	GET / PUT / DELETE	Get, update, or delete a specific news post
/api/admin/news/:id/publish	PATCH	Toggle publish/unpublish status
/api/admin/categories	GET / POST	List / Create categories
/api/admin/categories/:id	PUT / DELETE	Update / Delete category
/api/admin/hashtags	GET / POST	List / Create hashtags
/api/admin/hashtags/:id	PUT / DELETE	Update / Delete hashtag
/api/admin/cities	GET / POST	List / Create cities
/api/admin/cities/:id	PUT / DELETE	Update / Delete city
/api/admin/ads	GET / POST	List / Create advertisements
/api/admin/ads/:id	PUT / DELETE	Update / Delete ad
/api/admin/news-order	GET / POST	Get / Save news display order
/api/admin/notifications/send	POST	Send push notification
/api/admin/notifications	GET	List notification history
/api/admin/users	GET / POST	List / Create admin users
/api/admin/users/:id	PUT / DELETE	Update / Delete admin user
/api/admin/analytics	GET	Fetch all analytics metrics
/api/admin/s3/presign	POST	Get a pre-signed URL for S3 upload

7. FILE UPLOAD FLOW (AWS S3)

For all image and video uploads (news media, category images, ad banners, admin profiles):

- Step 1: Admin selects file(s) in the upload UI

- Step 2: Frontend calls POST /api/admin/s3/presign with filename and content type
- Step 3: Backend returns a pre-signed S3 URL
- Step 4: Frontend uploads the file directly to S3 using a PUT request to the pre-signed URL
- Step 5: Frontend stores the final S3 public URL and sends it as part of the create/update API call

 *Never upload files through the backend server. Always use direct S3 pre-signed URL upload.*

8. UI / UX REQUIREMENTS

- Clean professional design — dark sidebar, white main area
- Fully responsive (works on desktop, tablet)
- Use a consistent component system (shadcn/ui recommended)
- Toast notifications for success/error on all actions (e.g., 'News published successfully')
- Loading spinners during API calls
- Confirm dialogs (modal) before any destructive action (delete, unpublish)
- Empty states with helpful messages when tables have no data
- Pagination on all tables (10 or 20 rows per page)
- All forms have client-side validation with inline error messages

9. ROLE-BASED ACCESS CONTROL

- Super Admin: full access to all pages including Admin User Management
- Editor: can create, edit, publish news; can manage categories, hashtags, cities
- Reporter: can create news (draft only), cannot publish
- Hide/disable nav links and action buttons based on the logged-in user's role and permissions

 *Role and permissions are returned as part of the login API response. Store them in state and use them to conditionally render UI elements.*

10. FILE / FOLDER STRUCTURE (Recommended)

- /app or /pages — Route-based page components
- /components — Reusable UI components (DataTable, Modal, FormInput, etc.)
- /components/ui — shadcn/ui components
- /lib — Axios instance, API functions, utility helpers
- /store — Zustand/Redux store

- /hooks — Custom React hooks
 - /constants — API base URL, roles, colors
 - /public — Static assets
 - /.env.local — Environment variables (documented in .env.example)
-

11. ENVIRONMENT VARIABLES REQUIRED

- NEXT_PUBLIC_API_BASE_URL — Base URL of the REST API backend
- NEXT_PUBLIC_S3_BUCKET_URL — Public base URL of the S3 bucket for media

💡 *Never hard-code URLs or secrets. Use .env.local and document all variables in .env.example.*

12. DELIVERABLES

- Full Next.js / React.js project source code
- All pages and components as described in this document
- README with setup and run instructions
- .env.example with all required environment variables documented